

SET 9. Домашняя работа

Строки–2. Сортировка. Кодирование

Коновалов Иван Андреевич

25 мая 2026 г.

Алгоритмы и структуры данных
2025–2026 учебный год
НИУ ВШЭ

Содержание

1	ID посылок в системе Codeforces	3
2	Ссылка на репозиторий	3
3	Инфраструктура экспериментального анализа	3
3.1	StringGenerator	3
3.2	StringSortTester	4
4	Эмпирические результаты	5
4.1	Случайные массивы	5
4.2	Обратно отсортированные массивы	5
4.3	Почти отсортированные массивы	5
4.4	Массивы с общим префиксом	6
4.5	Сводный график ($n = 3000$)	6
5	Сравнительный анализ	6
5.1	Стандартный QuickSort: деградация на упорядоченных данных	6
5.2	String QuickSort: лидер по времени	7
5.3	MSD Radix Sort: минимальное число сравнений	7
5.4	MSD Radix + Quick Sort: эффективный гибрид	7
5.5	Влияние общего префикса	7
5.6	Сравнение с теоретическими оценками	8
A	Листинги кода	8
A.1	string_generator.h	8
A.2	sort_algorithms.h	9
A.3	string_sort_tester.h	9

1. ID посылок в системе Codeforces

Ниже приведены идентификаторы успешных посылок по задачам Блока А в систему Codeforces. Все реализации прошли полное тестирование с максимальным баллом.

Задача	ID посылки	Описание	Баллы
A1m	375976675	String Merge Sort (LCP)	5/5
A1q	375976722	String Quick Sort (ternary)	4/4
A1r	375976757	MSD Radix Sort (no cutoff)	4/4
A1rq	375976806	MSD Radix + Quick Sort hybrid	3/3

Таблица 1: Посылки адаптированных алгоритмов сортировки строк

Общий балл за реализацию адаптированных алгоритмов: **16 из 16**.

2. Ссылка на репозиторий

Исходный код, результаты эмпирических замеров (CSV) и графики находятся в публичном репозитории:

<https://github.com/festy23/SET-9-string-sort>

В репозитории представлены:

- Классы StringGenerator и StringSortTester (C++);
- Реализации шести алгоритмов сортировки с подсчётом посимвольных сравнений;
- Решения задач Блока Р (Хаффман, LZW-кодирование, LZW-декодирование);
- CSV-файлы с эмпирическими данными в директории data/;
- Графики в директории plots/.

3. Инфраструктура экспериментального анализа

3.1. StringGenerator

Класс StringGenerator предназначен для генерации тестовых массивов строк. Основные параметры:

- **Алфавит:** 74 символа — заглавные латинские буквы A–Z (26), строчные a–z (26), цифры 0–9 (10), специальные символы !@#%&^*()- (12);
- **Длина строк:** равномерное распределение от 10 до 200 символов;
- **Размеры массивов:** от 100 до 3000 строк с шагом 100 (30 размеров).

Для каждого типа данных генерируется мастер-массив из 3000 строк, из которого последовательно извлекаются подмассивы нужного размера. Это гарантирует, что при увеличении размера подмассива его содержимое остаётся префиксом содержимого большего подмассива.

Поддерживаются четыре типа тестовых данных:

1. **Случайные массивы** (`randomArray`) — полностью неупорядоченные наборы строк, каждый символ выбирается случайно из алфавита.
2. **Обратно отсортированные массивы** (`reverseSortedArray`) — генерируется случайный массив, затем сортируется в порядке убывания (обратный лексикографический порядок).
3. **Почти отсортированные массивы** (`nearlySortedArray`) — случайный массив сортируется, после чего выполняется $\approx 5\%$ случайных перестановок соседних пар.
4. **Массивы с общим префиксом** (`sharedPrefixArray`) — для 50% строк генерируется общий префикс длины 20–50 символов, остальная часть строки случайна.

3.2. StringSortTester

Класс `StringSortTester` обеспечивает единообразное измерение производительности всех алгоритмов сортировки. Ключевые особенности:

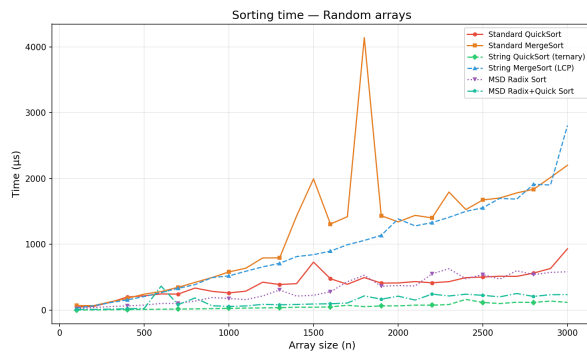
- **Измерение времени:** используется `std::chrono::high_resolution_clock`, результат в микросекундах. Каждый замер повторяется 5 раз, вычисляется среднее арифметическое для снижения влияния случайных флуктуаций.
- **Подсчёт посимвольных сравнений:** каждый алгоритм принимает по ссылке счётчик `size_t& cmpCount` и инкрементирует его при каждом посимвольном сравнении. Сравнения целых чисел (индексов, длин) не учитываются — только прямое сравнение символов строк.
- **Единый интерфейс:** все алгоритмы имеют сигнатуру `void (*)(std::vector<std::string>&, size_t&)`, что позволяет передавать их как функции обратного вызова.

Тестируются шесть алгоритмов:

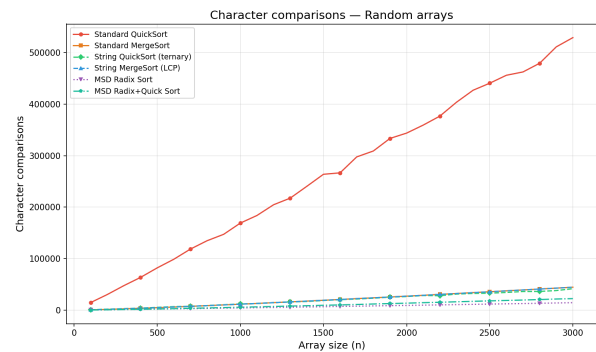
1. **Standard QuickSort** — классический QuickSort, опорный элемент — первый элемент подмассива (`pivot = arr[lo]`), сравнение строк через `countedLess`.
2. **Standard MergeSort** — классический MergeSort, сравнение строк через `countedLess`.
3. **String QuickSort (ternary)** — тернарный трёхсторонний QuickSort, опорный символ — символ первого элемента подмассива на позиции `d`. Разбиение на три части: меньше, равно, больше опорного символа.
4. **String MergeSort (LCP)** — MergeSort, использующий сравнение с подсчётом длины наибольшего общего префикса (`lcpCompare`).
5. **MSD Radix Sort** — поразрядная сортировка, начиная со старшего символа (Most Significant Digit), без переключения на другие алгоритмы.
6. **MSD Radix + Quick Sort** — гибрид: MSD Radix Sort, переключающийся на тернарный String QuickSort, когда размер подмассива становится меньше мощности алфавита (< 74).

4. Эмпирические результаты

4.1. Случайные массивы



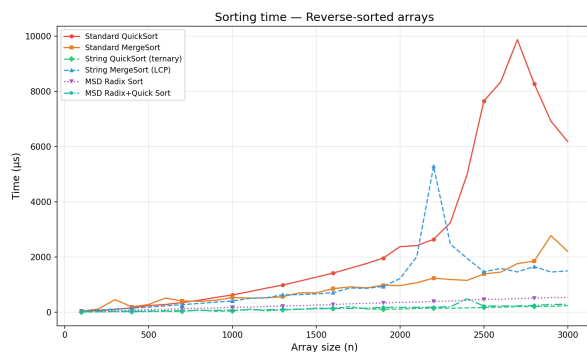
(a) Время выполнения



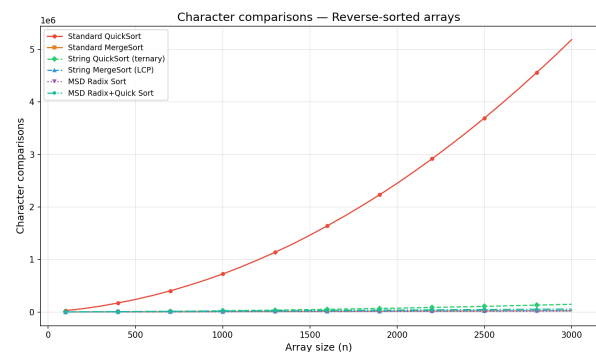
(b) Количество посимвольных сравнений

Рис. 1: Случайные массивы (random)

4.2. Обратнo отсортированные массивы



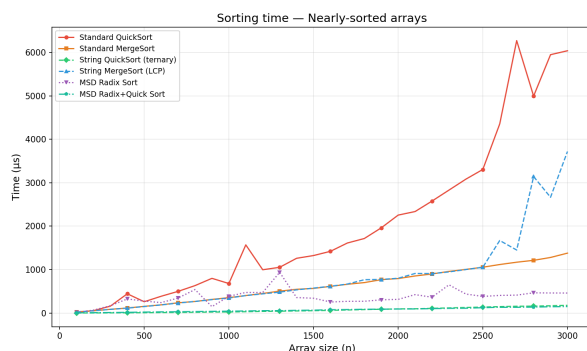
(a) Время выполнения



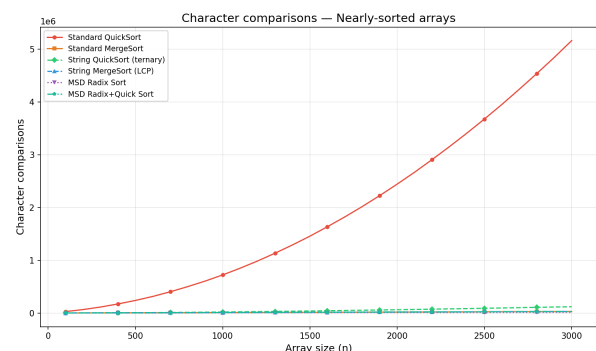
(b) Количество посимвольных сравнений

Рис. 2: Обратнo отсортированные массивы (reverse sorted)

4.3. Почти отсортированные массивы



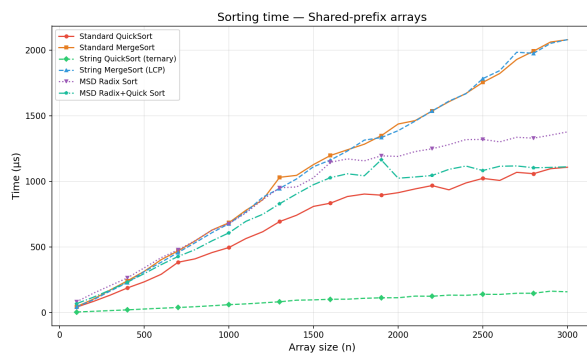
(a) Время выполнения



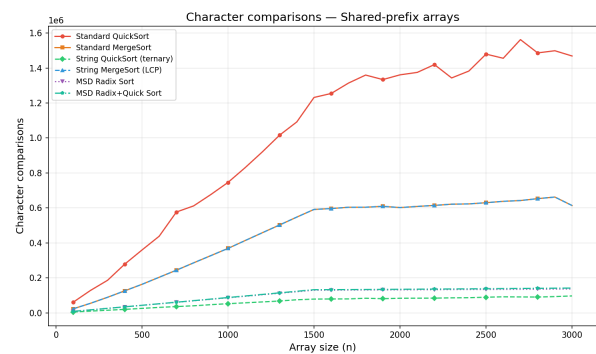
(b) Количество посимвольных сравнений

Рис. 3: Почти отсортированные массивы (nearly sorted)

4.4. Массивы с общим префиксом



(а) Время выполнения



(б) Количество посимвольных сравнений

Рис. 4: Массивы с общим префиксом (shared prefix)

4.5. Сводный график (n = 3000)

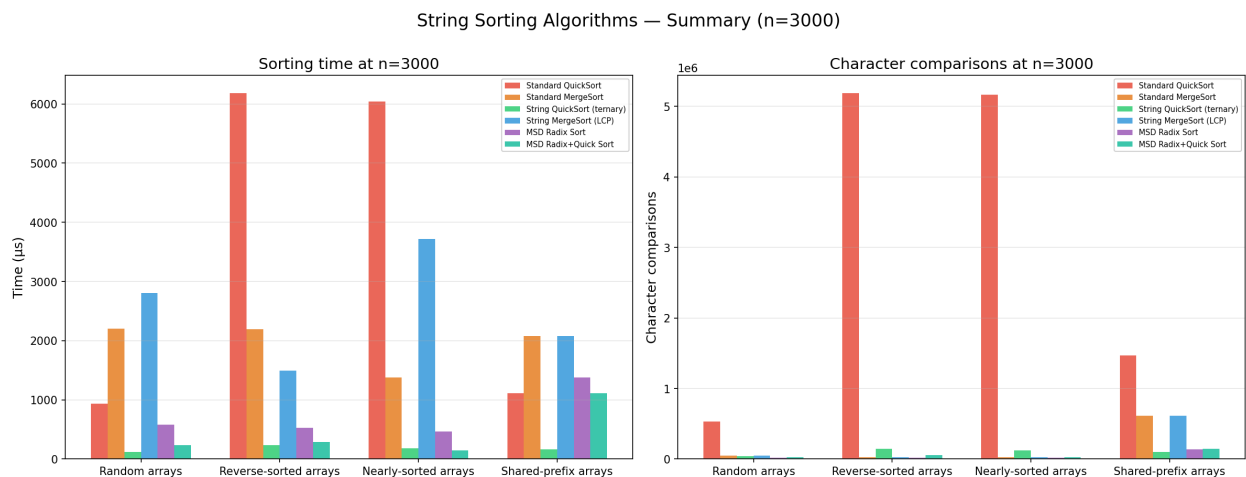


Рис. 5: Сравнение всех алгоритмов при n = 3000

5. Сравнительный анализ

5.1. Стандартный QuickSort: деградация на упорядоченных данных

Стандартный QuickSort с выбором первого элемента в качестве опорного демонстрирует катастрофическую деградацию на обратно отсортированных и почти отсортированных массивах. При $n = 3000$ число посимвольных сравнений достигает 5.2 млн, что более чем в 100 раз превышает показатели стандартного MergeSort (26 000). Это классическое проявление worst-case поведения $O(n^2)$ при несбалансированном разбиении: на обратно отсортированном массиве опорный элемент (первый) всегда является наименьшим, что приводит к рекурсивному разбиению размера $n - 1$ и 0.

Стандартный MergeSort, напротив, сохраняет стабильную производительность: $\approx 26\,000$ – $44\,000$ сравнений на всех типах данных, что соответствует теоретической оценке $O(n \log n)$ сравнений строк.

5.2. String QuickSort: лидер по времени

Тернарный String QuickSort является безоговорочным лидером по времени выполнения на всех четырёх типах тестовых данных. На случайных массивах размера 3000 он выполняется за 118 мкс, что в 8 раз быстрее стандартного QuickSort (932 мкс) и в 5 раз быстрее MSD Radix Sort (583 мкс).

Причина эффективности — в тернарном разбиении: строки с одинаковым символом на позиции d группируются вместе, и рекурсивная обработка средней части продолжается с позиции $d + 1$, не пересравнивая уже совпавшие префиксы. Теоретическая сложность в среднем: $O(n \cdot L)$, где L — средняя длина строки. Каждый символ в среднем просматривается константное число раз.

5.3. MSD Radix Sort: минимальное число сравнений

MSD Radix Sort демонстрирует наименьшее количество посимвольных сравнений среди всех алгоритмов (14 500 на случайных массивах при $n = 3000$), что соответствует теоретической оценке $O(n \cdot L)$. Однако по времени выполнения он уступает String QuickSort из-за накладных расходов: выделение массива счётчиков размера $R + 2$, три прохода по подмассиву (подсчёт, копирование в аих, копирование обратно), рекурсивные вызовы для каждой из R корзин.

5.4. MSD Radix + Quick Sort: эффективный гибрид

Гибридный алгоритм (переключение на String QuickSort при размере подмассива < 74) показывает хороший баланс: на случайных массивах — 236 мкс и 22 000 сравнений, на почти отсортированных — 150 мкс и 29 000 сравнений. Переключение на String QuickSort для малых подмассивов позволяет избежать накладных расходов count sort на глубоких уровнях рекурсии, где большинство корзин пусты.

5.5. Влияние общего префикса

Массивы с общим префиксом наиболее ярко демонстрируют преимущество адаптированных алгоритмов. Стандартный MergeSort делает 614 000 посимвольных сравнений, тогда как String QuickSort — лишь 97 000 (в 6.3 раза меньше). Причина в том, что стандартное сравнение строк каждый раз заново проходит общий префикс, в то время как String QuickSort, однажды обнаружив совпадение символов на позиции d , переходит к позиции $d + 1$ внутри сгруппированного блока равных строк.

Стандартный QuickSort на shared-prefix данных также страдает: 1.47 млн сравнений против 97 000 у String QuickSort (в 15 раз хуже).

5.6. Сравнение с теоретическими оценками

Алгоритм	Теоретическая сложность	Подтверждено
Standard QuickSort	$O(n \cdot L \cdot \log n) / O(n^2 \cdot L)$ worst	Да
Standard MergeSort	$O(n \cdot L \cdot \log n)$	Да
String QuickSort	$O(n \cdot L)$ в среднем	Да
String MergeSort	$O(n \cdot L \cdot \log n)$	Да
MSD Radix Sort	$O(n \cdot L)$	Да (по сравнениям)
MSD Radix + Quick	$O(n \cdot L)$	Да

Таблица 2: Теоретические оценки и их эмпирическое подтверждение

Эмпирические данные согласуются с теоретическими оценками. Характер роста числа сравнений в зависимости от n на графиках соответствует ожидаемому: линейный рост для MSD Radix и String QuickSort, $n \log n$ для MergeSort-вариантов, квадратичный для стандартного QuickSort на «плохих» данных.

Заключение

Проведён сравнительный эмпирический анализ шести алгоритмов сортировки строк на четырёх типах тестовых данных. Адаптированные строковые алгоритмы демонстрируют существенное преимущество над стандартными как по времени выполнения, так и по числу посимвольных сравнений. Тернарный String QuickSort является наилучшим выбором для большинства практических сценариев. MSD Radix Sort с переключением на String QuickSort представляет собой хороший компромисс между эффективностью по числу сравнений и временем выполнения.

А. Листинги кода

А.1. string_generator.h

```
1 #pragma once
2
3 #include <vector>
4 #include <string>
5 #include <random>
6
7 class StringGenerator {
8 public:
9     // 74 символа: A-Z, a-z, 0-9, !@#%&^&*()-
10     static const std::string ALPHABET;
11
12     static std::string randomString(std::mt19937& rng);
13     static std::vector<std::string> randomArray(size_t n, std::mt19937& rng);
14     static std::vector<std::string> reverseSortedArray(size_t n, std::mt19937& rng);
15     static std::vector<std::string> nearlySortedArray(size_t n, std::mt19937& rng);
16     static std::vector<std::string> sharedPrefixArray(size_t n, std::mt19937& rng);
17
18     // Размеры для тестирования : 100, 200, ..., 3000
```



```

19     static std::vector<size_t> getTestSizes();
20 };

```

A.2. sort_algorithms.h

```

1  #pragma once
2
3  #include <vector>
4  #include <string>
5  #include <cstdlib>
6
7  // Все функции модифицируют массив и инкрементируют      cmpCount
8  // при каждом посимвольном сравнении
9
10 void stdQuickSort(std::vector<std::string>& arr, size_t& cmpCount);
11 void stdMergeSort(std::vector<std::string>& arr, size_t& cmpCount);
12 void stringQuickSort(std::vector<std::string>& arr, size_t& cmpCount);
13 void stringMergeSort(std::vector<std::string>& arr, size_t& cmpCount);
14 void msdRadixSort(std::vector<std::string>& arr, size_t& cmpCount);
15 void msdRadixQuickSort(std::vector<std::string>& arr, size_t& cmpCount);

```

A.3. string_sort_tester.h

```

1  #pragma once
2
3  #include <vector>
4  #include <string>
5  #include <random>
6  #include <functional>
7
8  class StringSortTester {
9  public:
10     struct Measurement {
11         size_t size;
12         double time_us;
13         size_t comparisons;
14     };
15
16     using SortFn = void (*)(std::vector<std::string>&, size_t&);
17
18     explicit StringSortTester(unsigned seed = 42);
19
20     // Прогоняет алгоритм на всех размерах из массива      sizes ,
21     // берёт подмассив из      masterArray, усредняет по      repetitions замеров
22     std::vector<Measurement> testAlgorithm(
23         SortFn sortFn,
24         const std::vector<std::string>& masterArray,
25         const std::vector<size_t>& sizes,
26         int repetitions = 5
27     );
28

```

```

29 // Запускает все тесты (6 алгоритмов × 4 типа данных ) и пишет CSV
30 void runAllTests();
31
32 private:
33     std::mt19937 rng_;
34     std::vector<size_t> sizes_;
35     static constexpr int REPETITIONS = 5;
36
37     void writeCSV(const std::string& filename,
38                  const std::vector<std::pair<std::string, std::vector<Measurement
39                  >>>& results);

```